Decisions and Repetition

Engineering Computation with Python — Chapter 4

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Boolean Foundations
2. Decisions
3. Loops
4. Summary

Table of Contents

1 Boolean Foundations

2 Decisions

3 Loops

4 Summary

From Straight-Line Code to Real Decisions

Every program so far has run straight through, one line at a time. That is not enough for engineering work, which is full of:

Decisions

Is this reading within tolerance?

Repetition

Apply this check to all 101 samples.

The foundation beneath both

Every decision and every loop condition is built from Boolean expressions — expressions that evaluate to `True` or `False`.

Relational and Logical Operators

Symbol	Meaning
`<`	Less than
`<=`	Less than or equal
`==`	Equal
`!=`	Not equal
`>`	Greater than
`>=`	Greater than or equal

Logical	Elem.-wise	Short-circ.
AND	`&`	`and`
OR	`|`	`or`
NOT	`~`	`not`

Any nonzero value is `True`; zero is `False`.

© Munim Zabidi

Logical vs. Bitwise: Don’t Confuse Them

Logical operators (`and`, `or`, `not`) output `True`/`False`. Bitwise operators (`&`, `|`, `~`) work on the binary representation of numbers.

```
True and False  # False
True or False   # True
not True        # False

6 & 3  # 2  (Binary: 110 & 011 = 010)
6 | 3  # 7  (Binary: 110 | 011 = 111)
~5     # -6 (two’s complement)
```

A common mistake

Using `&` where you meant `and` compiles fine and gives a misleading numeric answer instead of an error.

Relational Operators in Action

```
a = 5
b = 10

print(a < b)     # True
print(a <= b)    # True
print(a == b)    # False
print(a != b)    # True
print(a > b)     # False
print(a >= b)    # False
```

Each of the six operators applied to the same pair of values — compare the results directly.

Operator Precedence

Python follows a strict precedence order across arithmetic, bitwise, and Boolean operators — from highest to lowest below:

Operator	Description
`**`	Exponentiation
`*`, `/`, `//`, `%`	Mult., div., floor div., remainder
`+`, `-`	Addition, subtraction
`&`, `^`, `|`	Bitwise AND, XOR, OR
`<`, `<=`, `>`, `>=`, `!=`, `==`	Comparisons
`not`, `and`, `or`	Boolean logic

When in doubt, parenthesise.

Precedence in Practice

```
a, b, c = 5, 2, 3

expr1 = (a + b) * c        # Parens first: 21
expr2 = a ** b + c         # Power first: 28
expr6 = a < b and b > c    # Relational, then AND: False
expr7 = a | b and c        # Bitwise OR then AND: 3
```

Expression 7 deserves a closer look

`a | b` evaluates first (bitwise OR: $5|2 = 7$). Since `and` returns one of its actual operands, not just `True`/`False`, the result is `c = 3`.

Combining All Three Kinds at Once

Real conditions usually combine more than one operator kind:

```
x, y, z = 8, 5, 12

result = ((x + y) > z) and ((x - y) * 2 < z) or (z % y == 0)
print(result)  # True

# (x + y) > z       : is the sum greater than z?
# (x - y) * 2 < z   : is double the difference less than z?
# z % y == 0        : does z divide evenly by y?
```

This is where precedence starts to matter — work through it piece by piece before trusting the printed result.

Chained Comparisons: Readable Range Checks

A range check written the traditional way:

```
x = 10
if x > 5 and x < 15:
    print("x is between 5 and 15")
```

can be written as a single chained comparison:

```
if 5 < x < 15:
    print("x is between 5 and 15")
```

Python evaluates left to right and combines the result with an implicit `and` — exactly the kind of test that decides whether a measurement lies within tolerance.

Chained Comparisons: More Examples

```
x = 15
print(10 < x < 20)          # True

y = 25
print(20 < y <= 30)         # True

a = 4
print(2 < a**2 < 20)        # True, 16 is between 2 and 20

name = "Bob"
print("Alice" < name < "Charlie")  # True (alphabetical)
```

Chained comparisons can involve more than two operators, mix data types, and even call functions — the chain is still evaluated left to right.

Table of Contents

1 Boolean Foundations

2 Decisions

3 Loops

4 Summary

Three Kinds of Statement

Program statements fall into three categories:

1. **Assignment** — `variable = expression`
2. **Decision** — executes statements if a condition is `True` (`if`, `if-else`, `if-elif-else`)
3. **Repetition** — executes statements many times (`while`, `for`)

This section covers decisions; the next covers repetition.

© Munim Zabidi

The if-elif-else Structure

```
if condition_1:
    statements
elif condition_2:
    statements
else:
    statements
```

Example

```
x = 2
if x < 0:
    print('neg')
elif x == 0:
    print('zero')
else:
    print('non-neg')   # prints: non-neg
```

Multiple Conditions with and

Logical operators combine multiple conditions in a single `if`. For $ax^2 + bx + c = 0$, equal roots exist when $b^2 - 4ac = 0$ and $a \neq 0$:

```
a = float(input('Input a value: '))
b = float(input('Input b value: '))
c = float(input('Input c value: '))

d = b**2 - 4 * a * c

if d == 0 and a != 0:
    x = -b / (2 * a)
    print(f'The value of x is: {x}')
else:
    print('Different root')
```

Multiple Conditions with if-elif-else

Three outcomes: $d > 0$ (two real roots), $d = 0$ (one root), $d < 0$ (complex roots) — more than two conditions needs `elif`:

```
d = b**2 - 4 * a * c

if d == 0 and a != 0:
    x1 = -b / (2 * a)
    print(f'The roots are both: {x1}')
elif d > 0 and a != 0:
    x1 = (-b + (d**0.5)) / (2 * a)
    x2 = (-b - (d**0.5)) / (2 * a)
    print(f'The roots are: {x1} and {x2}')
else:
    print('Complex root')
```

Nested if: When One Check Depends on Another

Check $a \neq 0$ first; only then does the discriminant calculation make sense (if $a = 0$, the equation is linear, not quadratic):

```
if a != 0:
    if d == 0:
        x1 = -b / (2 * a)
        print(f'The roots are both: {x1}')
    elif d > 0:
        x1 = (-b + (d**0.5)) / (2 * a)
        x2 = (-b - (d**0.5)) / (2 * a)
        print(f'The roots are: {x1} and {x2}')
    else:
        print('Complex root')
else:
    x1 = -c / b
    print(f'The root is: {x1}')
```

Table of Contents

1 Boolean Foundations

2 Decisions

3 Loops

4 Summary

Two Loop Constructs

`for`: determinate

Repeats for every value in a sequence (e.g. `range`) — the number of iterations is known before the loop starts.

```
for i in range(start, stop, step):
    statements
```

`while`: indeterminate

Repeats as long as a condition is true — the number of iterations is not necessarily known in advance.

```
while condition:
    statements
```

`range(j, k)`: generates j to $k - 1$. `range(j, k, m)`: step m.

© Muním Zabidi

Worked Example: Compound Interest with for

$$B = a(1 + r)^n$$

Evaluate B for $a = \$100$, $r = 8\%$, over $n = 2, 4, 6, 8, 10$:

```python
a = 100   # initial investment
r = 0.08  # annual interest rate

for n in range(2, 11, 2):
    B = a * (1 + r) ** n
    print(f'n = {n}, B = {B:.2f}')
```

```
n = 2, B = 116.64
n = 4, B = 136.05
...
n = 10, B = 215.89
```

Recognizing Σ as a Loop

Your mathematics courses write $\sum_{i=1}^{5} i = 15$. A `for` loop is precisely this idea, written one step at a time:

Notation	Meaning	Python
(implicit)	Total starts at 0	`total_sum = 0`
$i = 1$ (below Σ)	Start index	`range(1, ...`
5 (above Σ)	End index (incl.)	`..., 6)`
summed term	Added each step	`total_sum += i`

The Summation Pattern

```
total_sum = 0
for i in range(1, 6):      # i = 1, 2, 3, 4, 5
    total_sum += i
print(total_sum)           # 15
```

The same skeleton, every time

$\sum i^2$ or $\sum 1/i$ use the exact same loop — only the expression added to the running total changes. Recognizing Σ as "a `for` loop with a running total" is one of the most useful shortcuts in this course.

The Same Computation with while

```
a = 100; r = 0.08
n = 2

while n <= 10:
    B = a * (1 + r) ** n
    print(f'n = {n}, B = {B:.2f}')
    n += 2   # must increment manually
```

The key difference from for

`while` requires `n` to be initialized and manually incremented — the number of iterations is governed entirely by the condition.

Worked Example: Capacitor Charging

$$Q(t) = CV\left(1 - e^{-t/RC}\right)$$

Monitor charge every 0.5 s until it exceeds 2 units ($V = 9, R = 4, C = 1$) — the number of steps is not known in advance, so `while` is the natural choice:

```
import math
V, R, C = 9, 4, 1
t, q = 0, 0
Q, T = [], []

while q <= 2:
    q = C * V * (1 - math.exp(-t / (R * C)))
    Q.append(q); T.append(t)
    t += 0.5
```

Two Ways to Grow a List

.append()

```
Q.append(q)
T.append(t)
```

+=

```
Q += [round(q, 4)]
T += [t]
```

```
Q: [0.0, 1.0575, 1.9908, 2.8144]
T: [0, 0.5, 1.0, 1.5]
```

Both produce identical results — a matter of style.

Nested Loops: Two Values at Once

B for 3 values of *a* and 5 values of *n* needs two loops, one inside the other:

```
a_values = [100, 500, 800]
n_values = [2, 4, 6, 8, 10]
r = 0.09

for a in a_values:        # outer loop
    for n in n_values:    # inner loop
        B = a * (1 + r) ** n
        print(f"a={a}, n={n}, B={B:.2f}")
```

The inner loop runs to completion for *every* value of the outer loop — $3 \times 5 = 15$ total iterations.

continue: Skip This Iteration

`continue` skips the rest of the current iteration and moves to the next one.

```
for n in range(1, 51):
    if n % 7 != 0:
        continue          # skip non-multiples of 7
    print(f"Divisible by 7: {n}")
```

```
Divisible by 7: 7
Divisible by 7: 14
...
Divisible by 7: 49
```

In nested loops, `continue` only skips the inner loop it occurs in.

break: Exit the Loop Entirely

`break` terminates the loop immediately — useful for an indeterminate loop that should stop as soon as a goal is met.

```
import random
x = random.randint(1, 6)
n = 0

while True:                # loop forever...
    guess = int(input('Guess the number: '))
    n += 1
    if guess == x:
        print(f'Correct after {n} tries!')
        break              # ...until the guess is right
```

break vs. continue

break

Terminates the loop entirely. Statements after `break` inside the loop do not execute.

continue

Skips to the next iteration. Remaining statements in *this* iteration are skipped.

A caution

Both are useful, but a loop whose logic depends heavily on them is often clearer when rewritten without them.

Table of Contents

1 Boolean Foundations

2 Decisions

3 Loops

4 Summary

What You Should Take Away

- **Relational operators** return `True`/`False`; these Boolean values are what every decision and loop condition is built from.
- **Logical** (`and`, `or`, `not`) vs. **bitwise** (`&`, `|`, `~`) — confusing the two is a frequent source of error.
- Python evaluates by strict **precedence** — arithmetic, then comparison, then logical. Parenthesise when in doubt.
- **Chained comparisons** (`5 < x < 15`) express range checks in one readable line.

What You Should Take Away (continued)

- **if-elif-else** selects which block runs; an `elif` chain stops at the first true condition, so order matters.
- **for** runs a known number of times over a sequence; **while** runs until a condition is false — the right choice when iterations are not known in advance.
- `break` exits a loop immediately; `continue` skips to the next iteration.
- A Σ in a formula is "a `for` loop with a running total" — one of the most useful shortcuts in this course.

Next: Functions and Reusable Calculations